TRƯỜNG ĐẠI HỌC PHENIKAA

KHOA HỆ THỐNG THÔNG TIN

BÁO CÁO BÀI TẬP LỚN — CSE703093: AN TOÀN PHẦN MỀM

PHƯƠNG PHÁP KIỂM THỬ THEO FUZZING CHO HỆ THỐNG BÁN HÀNG

Giảng viên: ThS. Vũ Quang Dũng

STT	Họ và tên	Mã số sinh viên	Lớp
1	Nguyễn Thị Huyền Trang	23010181	CNTT2
2	Nguyễn Thị Quỳnh Anh	23010147	CNTT2
3	Trần Xuân Thành	23010160	CNTT2

Năm học: 2026 - 2027

MỤC LỤC

1. Giới thiệu, mục tiêu và phạm vi 4
1.1. Bối cảnh và vấn đề an toàn phần mềm 4
1.2. Mục tiêu và câu hỏi thực nghiệm 4
1.3. Phạm vi và tiêu chí kết luận 4
2. Cơ sở lý thuyết và hệ thống mục tiêu 5
2.1. Liên hệ kiến thức bài giảng 5
2.2. Kiến trúc thực nghiệm và dữ liệu nền 5
2.3. Năm luồng nghiệp vụ được khảo sát 5
3. Đặc tả, testcase và phương pháp fuzzing 7
3.1. Tổ chức đặc tả và phạm vi tự động hóa 7
3.2. Seed, mutation và blackbox 8
3.3. Khảo sát bằng coverage và chuyển giao input 8
3.4. Whitebox concolic có giới hạn bằng Z3 8
3.5. Oracle, dữ liệu sau lỗi và mức bằng chứng 9
4. Kết quả thực nghiệm và phân tích 11
4.1. Cấu hình và nguồn bằng chứng 11
4.2. Campaign theo 5 Excel đã đồng bộ 11
4.3. Kết quả blackbox, coverage và concolic 11
4.4. Phân tích phản ví dụ theo luồng nghiệp vụ 12
4.5. Kiểm tra độ đúng của bộ công cụ 13
5. Thảo luận, hạn chế và hướng phát triển 14
5.1. Mức đáp ứng theo nhóm lỗi 14
5.2. Đe dọa tính hợp lệ của kết luận 14
5.3. Hướng khắc phục và phát triển 14
6. Kết luận 15
Tài liệu tham khảo 16
Phụ lục A. Cấu hình, lệnh chạy và bằng chứng 17
A.1. Chuẩn bị và chạy lại 17
A.2. Chú ý khi tái hiện và sử dụng công cụ 17
Phụ lục B. Truy vết 29 testcase API đã chạy 19

DANH MỤC BẢNG

Bảng 1. Kiến thức áp dụng và phạm vi triển khai 5
Bảng 2. Module, luồng và tài sản cần bảo vệ 5
Bảng 3. Số testcase thiết kế và API có kịch bản 7
Bảng 4. Ví dụ truy vết yêu cầu, input và oracle 7
Bảng 5. Miền tượng trưng và đại diện gửi qua API 8
Bảng 6. Quy ước thống kê và kết luận 10
Bảng 7. Các chiến dịch hiện tại được báo cáo 11
Bảng 8. Kết quả testcase API theo module 11
Bảng 9. Kết quả demo theo giai đoạn, không gộp lỗi gốc 11
Bảng 10. Các input concolic được đo 12
Bảng 11. Thiết kế và mức thực nghiệm hiện tại 14
Bảng 12. Bản đồ bằng chứng đang tồn tại 17
Bảng 13. Danh sách testcase API và kết quả theo ID 19

1. Giới thiệu, mục tiêu và phạm vi

1.1. Bối cảnh và vấn đề an toàn phần mềm

Hệ thống bán hàng tiếp nhận dữ liệu từ client tại các thao tác đăng nhập, tạo đơn, sửa hồ sơ, quản lý sản phẩm và cập nhật trạng thái. Dữ liệu đúng cú pháp JSON vẫn có thể sai kiểu, sai miền giá trị hoặc cố ý vượt quyền. Nếu server tin cậy giá client, cho cập nhật toàn bộ body hoặc bỏ qua trạng thái trước đó, hệ thống có thể mất tính toàn vẹn mà không phát sinh crash.

Theo Chương 1, nhóm lỗi an toàn phần mềm gồm an toàn bộ nhớ, kiểm tra đầu vào, số học, đồng bộ hóa, xử lý lỗi/ngoại lệ và logic nghiệp vụ; xác thực, phân quyền và bảo vệ thông tin cũng là các nội dung liên quan. Vì vậy đề tài giữ cả oracle về quyền và toàn vẹn dữ liệu, thay vì chỉ tìm lỗi bộ nhớ [1, trang 14, 41, 49, 53].

1.2. Mục tiêu và câu hỏi thực nghiệm

- Xây dựng đặc tả và testcase truy vết được: yêu cầu → điều kiện/input → oracle → bằng chứng.
- Tự động khám phá input bất thường bằng mutation từ seed hợp lệ, lưu và tái hiện các phản ví dụ.
- Phân biệt blackbox, khảo sát có phản hồi coverage và whitebox concolic/SMT theo Chương 4.
- Triển khai concolic có chọn lọc trên các điều kiện xử lý totalAmount thật của API mô phỏng; xác nhận model bằng lần chạy cụ thể.
- Đánh giá trong phạm vi đã chạy, không đồng nhất finding occurrences với lỗi gốc, HTTP 500 với crash hoặc Excel đầy đủ với chứng minh an toàn toàn hệ thống.

Các câu hỏi chính là: mutation có tạo được phản ví dụ ngoài seed không; oracle nào nhận biết vi phạm; input được chuyển sang khảo sát có tái hiện từ dữ liệu sạch không; Z3 có sinh được input đạt tiền tố nhánh đã giải không? Báo cáo không đặt mục tiêu chứng minh mọi lỗ hổng đã được tìm thấy hoặc phương pháp có solver luôn nhanh hơn.

1.3. Phạm vi và tiêu chí kết luận

Đối tượng thực nghiệm là node_harness: Express xử lý request thực, datastore trong bộ nhớ và dịch vụ mô phỏng. Các kết quả được đo khi thực thi, không phải bảng kết quả điền sẵn. Kết luận áp dụng cho hành vi của harness ở phiên bản đã lưu hash, chưa phải kết luận cho MongoDB hoặc hệ thống production.

Năm Excel giữ 10 sheet/file và 104 testcase. Campaign hiện có kịch bản cho 29 testcase API; 75 testcase còn lại được ghi riêng là ngoài campaign. Mẫu C/Python, mô hình CTL/số học, timing, bộ đếm và các kịch bản race/version chưa được campaign này kiểm chứng lại. Concolic bổ sung cho TC-ORD-3/4, không tạo thêm ID hoặc tự triển khai các testcase còn lại.

2. Cơ sở lý thuyết và hệ thống mục tiêu

2.1. Liên hệ kiến thức bài giảng

Bảng 1. Kiến thức áp dụng và phạm vi triển khai

Nội dung	Cách áp dụng	Giới hạn
Chương 1: CIA, quyền và trust boundary	Bảo vệ tài khoản; ngăn sửa giá, role và trạng thái trái phép	Đánh giá ảnh hưởng trong harness, chưa chấm CVSS production
Chương 4: seed, mutation và oracle	Sinh input bất thường; kiểm tra response và dữ liệu lưu	Ngân sách hữu hạn, oracle chỉ bao phủ điều kiện đã cài
Coverage feedback	Giữ input mở thêm dòng trong index.js	Có setup; chưa là branch/edge coverage toàn hệ thống
Concolic + SMT	Trace nhánh totalAmount, phủ định tiền tố, Z3 và chạy xác nhận	Mô hình kiểu thủ công, tám đại diện JSON
Crash/hang/sanitizer	Phân biệt mức bằng chứng và các giá trị chưa đo	Chưa có sanitizer xác nhận lỗi bộ nhớ native

Bài giảng cho phép dùng invariant để phát hiện lỗi logic không gây crash [2, trang 10–11, 24]. Coverage là tín hiệu chọn input, không phải chứng minh không còn lỗi [2, trang 35, 41]. Whitebox trong Chương 4 dùng symbolic/concolic execution và SMT, do đó cần phân biệt với chế độ chỉ dùng coverage [3, trang 7–9, 45, 53–54].

2.2. Kiến trúc thực nghiệm và dữ liệu nền

```
Excel + seed_corpus → Python sinh input / điều phối
  → HTTP hoặc Node harness → Express → datastore mô phỏng
  → response + dữ liệu thay đổi + coverage / trace nhánh
  → oracle → report.json + evidence + snapshot/hash
```

Nguồn dữ liệu nền nằm trong node_harness/models.js: tài khoản user/admin/victim, sản phẩm và các đối tượng mô phỏng. Seed request được lưu tại seed_corpus/*.json. testcases.py mô tả kịch bản thực thi; spec_campaign.py đọc workbook, liên kết ID, điều phối và tổng hợp. Không tự chuyển mọi câu yêu cầu tự nhiên trong Excel thành code.

Trust boundary chính là client → API, token → quyền tài nguyên, body → dữ liệu lưu và giá client → catalog phía server. Hợp đồng mong đợi yêu cầu xác minh quyền và dữ liệu trước khi lưu; code mục tiêu có thể thiếu các kiểm soát đó. Mô tả luồng mong đợi không phải khẳng định ứng dụng đã thực hiện đầy đủ.

2.3. Năm luồng nghiệp vụ được khảo sát

Bảng 2. Module, luồng và tài sản cần bảo vệ

Module	Luồng nghiệp vụ	Rủi ro chính
AUTH	Email/password → tìm user → bcrypt → JWT	Sai kiểu, lộ tài khoản, role không tin cậy
ORD	Token + giỏ/địa chỉ → kiểm tra → lưu đơn	Giỏ sai cấu trúc, tiền sai kiểu, giá client giả

Module	Luồng nghiệp vụ	Rủi ro chính
OS	Đơn hợp lệ → yêu cầu trạng thái → lưu/thông báo	Vượt quyền, enum sai, phá trạng thái kết thúc
USR	Người gọi + user đích → lọc trường → lưu hồ sơ	IDOR/BOLA, mass assignment, mật khẩu dạng rõ
PRD	Thao tác quản trị → kiểm tra ID/giá → lưu	Thiếu quyền, giá sai, ID trùng, tài nguyên không tồn tại

Blackbox luân phiên bảy target, trong đó có login. Khảo sát coverage có sáu target: orders, order_status, user_update_self/victim, product_create/update; login phục vụ lấy token, chưa là target khảo sát độc lập. Concolic chỉ tập trung POST /api/orders và trường totalAmount.

3. Đặc tả, testcase và phương pháp fuzzing

3.1. Tổ chức đặc tả và phạm vi tự động hóa

Các sheet ThongTin, YeuCauChucNang, YeuCauPhiChucNang, RangBuoc và TestCase mô tả hợp đồng và điều kiện kiểm tra. Các sheet MoHinhTrangThai, ThuocTinhAnToan, LoaiTruLanNhau, HamSinhMa, KiemTraSoHoc được giữ để trình bày mô hình bổ sung; chưa coi chúng đã được bộ công cụ hiện tại kiểm chứng. Cấu trúc Excel được tham khảo theo hướng dẫn học phần [4, 5].

Bảng 3. Số testcase thiết kế và API có kịch bản

Module	Trong Excel	API đã triển khai	Ngoài campaign
AUTH	19	5	14
ORD	21	8	13
OS	22	6	16
USR	21	5	16
PRD	21	5	16
Tổng	104	29	75

ID và schema bảy cột TestCase được giữ nguyên. Testcase được ưu tiên theo đầu vào/quyền/trạng thái, tái hiện và phân loại, mẫu ngoại lệ/bộ nhớ, đồng bộ/timing/số học và seed đối chứng. KetQuaMongDoi mô tả điều kiện an toàn, không chứa kết luận thực nghiệm cũ. Kết quả từng lần chạy lưu riêng trong results/ và liên kết bằng snapshot/hash.

Các điều kiện bổ sung trong Excel chưa được mọi oracle tự động kiểm tra đầy đủ. Ví dụ, một kịch bản kiểm tra status chưa tự chứng minh không có mọi tác dụng phụ hoặc rò rỉ thông tin. ThongTin của từng module ghi rõ phạm vi này. Yêu cầu an toàn không được sửa theo hành vi sai chỉ để testcase đạt.

Bảng 4. Ví dụ truy vết yêu cầu, input và oracle

Testcase	Input/tình huống	Oracle chính
TC-AUTH-3	Không có tài khoản và sai mật khẩu	So sánh phản hồi công khai
TC-ORD-3/4	Thiếu/null/sai kiểu totalAmount	HTTP; concolic thêm trace và orders_created
TC-ORD-18	Giá 1, quantity 2, tổng 2; catalog 200000	Từ chối hoặc tổng theo catalog 400000
TC-OS-21	Hủy thành công rồi yêu cầu giao	HTTP và trạng thái lưu sau request
TC-USR-2/4	Role do client đặt; cập nhật password	Role và password_audit
TC-PRD-5	Tạo sản phẩm dùng ID đã tồn tại	Từ chối hoặc định danh mới duy nhất

3.2. Seed, mutation và blackbox

Seed là request hợp lệ dùng làm điểm xuất phát. Hạt giống ngẫu nhiên điều khiển lựa chọn mutation giúp tái tạo chuỗi lựa chọn trong cùng phiên bản và thứ tự chạy; không bảo đảm thời gian, token hoặc mọi ID môi trường giống nhau. Blackbox không dùng coverage để chọn input.

mutate_json() sao chép seed rồi chọn một mutation: thay giá trị trường, xóa trường, thay kiểu, sửa trường lồng hoặc nhân bản phần tử mảng. Dictionary gồm null, rỗng, chuỗi dài, số âm/lớn, boolean, object, array, Unicode và token có dạng tấn công. Đây là mutation cấu trúc JSON; chưa triển khai đầy đủ bit/byte flip, havoc, splicing hoặc grammar-based generation như các kỹ thuật mở rộng trong bài giảng.

run_campaign() thiết lập token, dữ liệu và target; gửi request, áp oracle rồi lưu target, input, had_auth, response và loại lỗi. Chèn token SQL/XSS vào dictionary không đồng nghĩa đã xác minh SQL injection/XSS: cần đúng sink và oracle. Blackbox giữ trạng thái suốt campaign; thay đổi sản phẩm có thể ảnh hưởng request sau. Danh mục bất thường được ghi riêng để tránh dùng catalog hỏng làm oracle giả đáng tin.

3.3. Khảo sát bằng coverage và chuyển giao input

Chế độ cũ trong whitebox_fuzzer.py là coverage-guided/greybox. fuzz_target() dùng hàng đợi riêng cho mỗi target, lấy input cha từ cuối hàng đợi, đột biến tối đa ENERGY=6 lần và giữ input con khi mở được dòng mới. Hết hàng đợi quay lại seed; tổng lượt được giới hạn đúng ngân sách.

Với --blackbox-report, focus_blackbox() chọn tối đa max-cases ứng viên có target, body và had_auth được hỗ trợ. Lần đầu là exact_replay; sau đó neighbor_probe thử x−1/x+1/0 hoặc thay chiều dài; các lượt còn lại dùng coverage_mutation. Không tự đoán auth thiếu. Thực thi nguyên body và trạng thái bật/tắt auth không có nghĩa khôi phục cùng token, ID hoặc lịch sử dữ liệu.

Node Inspector và v8-to-istanbul thu dòng được chạy trong node_harness/index.js. Phép đo có cả khởi tạo/setup, không đo toàn bộ models.js, middleware, datastore hoặc server production. Không công bố tỷ lệ bao phủ toàn hệ thống khi thiếu mẫu số phù hợp; không coi tập dòng là branch/edge coverage. Mỗi input chạy trong Node mới: thuận lợi cô lập nhưng có chi phí khởi động và có thể khác trạng thái blackbox.

3.4. Whitebox concolic có giới hạn bằng Z3

Phần concolic liên kết trực tiếp với POST /api/orders → OrderModel.save → castNumberOrThrow(totalAmount) trong datastore mô phỏng. branch_trace.js ghi điều kiện được chọn mà không đổi kết quả điều kiện; tracing chỉ bật khi single_run_harness nhận trace_branches. Không cấy thêm nhánh lỗi hoặc sửa ứng dụng để tạo kết quả mong muốn.

Bảng 5. Miền tượng trưng và đại diện gửi qua API

total_kind	Loại	Giá trị cụ thể
0	number	400000

total_kind	Loại	Giá trị cụ thể
1	numeric_string	"400000"
2	invalid_string	"not-a-number"
3	object	{}
4	null	null
5	missing	Xóa trường totalAmount
6	boolean	true
7	array	[]

Biến tượng trưng total_kind là Int trong [0,7]. predicates() mô hình hóa thủ công các guard required, empty, number, boolean và invalid cho tám đại diện. Các trường còn lại giữ seed hợp lệ. Nhánh kiểm tra Number.isFinite không được mô hình hóa tổng quát vì đại diện number hiện là số hữu hạn. Không tuyên bố phân tích mọi số thực/chuỗi hoặc tự symbolic JavaScript.

```
1. Chạy seed hợp lệ 400000, lấy branch_trace cụ thể.
2. Với một nhánh: giữ tiền tố, phủ định nhánh; bỏ hậu tố cũ.
3. Z3 giải miền + tiền tố mới → SAT/model, UNSAT hoặc unknown.
4. Giải mã model thành JSON; gửi lại POST /api/orders.
5. validate_trace kiểm tra mô hình và tiền tố khớp nhánh thực chạy.
6. Lưu evidence; replay input có vi phạm trong Node mới.
```

solve_flip() đặt timeout solver 2000 ms và phân biệt unknown với UNSAT. Nếu thiếu trace, lệch mô hình hoặc không đạt tiền tố mục tiêu, công cụ báo lỗi thay vì xác nhận nhánh giả. Input theo model đã chạy có thể được dùng lại cho một tiền tố tương ứng; report lưu chỉ số execution để truy vết.

Ví dụ SAT: seed số làm order.total.required=false. Phủ định điều kiện này dẫn đến kind=4 hoặc kind=5; solver có thể chọn null hoặc missing. API được chạy lại với đại diện cụ thể để xác nhận required=true. Solver không bắt buộc chọn cả hai trong một campaign.

```
Ví dụ UNSAT trong miền đại diện:
Tiền tố: NOT(kind = 4 OR kind = 5)  [đã qua required]
Nhánh muốn đạt: kind = 4 OR kind = 5  [cast.empty = true]
Hai ràng buộc mâu thuẫn → UNSAT.
```

UNSAT chỉ áp dụng cho công thức và miền được mã hóa, không chứng minh hệ thống không còn lỗi. Vòng lặp hiện là concolic có chọn lọc trên một vùng, chưa tự phát hiện coverage plateau hoặc trao đổi corpus hai chiều như hybrid fuzzing kiểu Driller. Cũng không khẳng định SMT nhanh hơn duyệt đủ tám đại diện trong ví dụ nhỏ này [3, trang 51–54].

3.5. Oracle, dữ liệu sau lỗi và mức bằng chứng

oracles.py hiện kiểm tra status, phản hồi đăng nhập, JWT, role, catalog total, dữ liệu đơn, trạng thái kết thúc, mật khẩu và ID trùng. Bộ đếm/version/timing không còn là oracle thực thi của campaign hiện tại. fuzzer_common.py cung cấp các kiểm tra response cho blackbox và khảo sát coverage.

Concolic đo thêm orders_created trước/sau request. Seed number phải tạo đơn thành công để phép thử có ý nghĩa; nếu seed lỗi thì dừng đánh giá. Request bị từ chối nhưng tạo đơn được ghi STATE_CHANGED_AFTER_REJECTION. Kiểu khác number được chấp nhận ghi INVALID_NUMERIC_TYPE_ACCEPTED theo hợp đồng API; đây chưa tự chứng minh khả

năng khai thác hoặc thiệt hại tài chính. Input có vi phạm được chạy lại và so HTTP, trace và số đơn được tạo, không coi đây là so sánh toàn bộ response hoặc dedup stack trace.

Bảng 6. Quy ước thống kê và kết luận

Quan sát	Kết luận được phép
PASS / FAIL	Thỏa/vi phạm oracle trên mẫu đã chạy; không là chứng minh toàn yêu cầu
ERROR	Setup, tiến trình hoặc đánh giá lỗi; chưa gán kết luận nghiệp vụ
Ngoài campaign	Có thiết kế trong Excel nhưng chưa được campaign này thực thi
HTTP 5xx	APPLICATION_HTTP_ERROR; chưa phải process crash
TIMEOUT	SUSPECTED_HANG; cần loại trừ setup/kết nối và kiểm tra tiến trình
Sanitizer / crash	Cần chẩn đoán vị trí lỗi hoặc log/mã thoát/signal phù hợp
Finding occurrence	Lượt quan sát vi phạm, chưa là số lỗi gốc độc lập
Null ở chỉ tiêu chưa đo	Chưa có bằng chứng đo; không phải 0 lỗi

runtime.py kiểm tra marker kết quả, JSON, định danh môi trường, mã thoát và timeout. Artifact JSON được ghi vào file tạm trước khi thay kết quả. Campaign lưu source_sha256 và spec_snapshot.json; concolic lưu các nguồn/seed được chọn cùng phiên bản Z3. Đây là truy vết phiên bản, chưa phải ghi lại toàn bộ thực thi như record-and-replay mức hệ điều hành.

4. Kết quả thực nghiệm và phân tích

4.1. Cấu hình và nguồn bằng chứng

Bảng 7. Các chiến dịch hiện tại được báo cáo

Chiến dịch	Cấu hình	Nguồn từ fuzzing-security-testing/
Theo đặc tả	Seed 7; fuzz-cases=8; 29 ID/59 kịch bản	results/spec_excel_aligned/
Demo blackbox + coverage	42 input; max-cases=2; 8 lượt/ứng viên	results/demo_chuong4_final/
Concolic	Budget 8; miền tám đại diện; Z3 5.1.0	results/demo_chuong4_final/concolic/

Kết quả lịch sử 48 testcase, so sánh ba seed hoặc kiểm chứng CTL/SMT trước đây không được gộp vào các bảng hiện tại. Các thư mục/công cụ đã đưa ra khỏi bộ chính không được liệt kê như lệnh tái hiện đang chạy được. Một bản báo cáo hoàn chỉnh không có nghĩa mọi testcase thiết kế đã hoàn thành.

4.2. Campaign theo 5 Excel đã đồng bộ

Bảng 8. Kết quả testcase API theo module

Module	Đã chạy	PASS	FAIL
AUTH	5	4	1
ORD	8	2	6
OS	6	1	5
USR	5	1	4
PRD	5	1	4
Tổng	29	9	20

Theo kịch bản, 59 lượt gồm 19 PASS và 40 FAIL, không có ERROR bộ chạy. 75 testcase ngoài campaign được ghi lý do trong excluded_testcases.json; chúng không vào mẫu số PASS/FAIL của 29 testcase. Tỷ lệ PASS 9/29 khoảng 31,03% chỉ mô tả tập mẫu và oracle này, không phải điểm an toàn của phần mềm hoặc hiệu quả fuzzing.

Một testcase nhiều kịch bản có thể FAIL chỉ vì một kịch bản vi phạm. Không dùng 59 kịch bản làm mẫu số cho tỷ lệ testcase, không cộng 40 kịch bản FAIL thành 40 lỗ hổng. Các điều kiện đặc tả mở rộng cần được đo riêng nếu oracle chưa kiểm tra.

4.3. Kết quả blackbox, coverage và concolic

Bảng 9. Kết quả demo theo giai đoạn, không gộp lỗi gốc

Giai đoạn	Input mới	HTTP 5xx	Quan sát vi phạm
Blackbox	42	1	22 finding occurrences
Khảo sát coverage	16	6	6 finding occurrences
Concolic	5	2	4 input vi phạm oracle

Concolic chạy thêm bốn lượt replay. Bốn input vi phạm tái hiện cùng HTTP, trace và số đơn được tạo; chưa nhóm theo stack hoặc xác nhận số lỗi gốc. Có một truy vấn UNSAT. Chiến dịch

dừng sau khi không còn ứng viên mới theo các đường đại diện đã chọn, không có nghĩa thử mọi giá trị của trường totalAmount.

Bảng 10. Các input concolic được đo

Đại diện	Nguồn	HTTP	Kết luận theo oracle
number	valid_seed	200	Không quan sát vi phạm trong mẫu
null	solver_generated	500	APPLICATION_HTTP_ERROR
numeric_string	solver_generated	200	INVALID_NUMERIC_TYPE_ACCEPTED
boolean	solver_generated	200	INVALID_NUMERIC_TYPE_ACCEPTED
object	solver_generated	500	APPLICATION_HTTP_ERROR

Chiến dịch concolic ghi 503 dòng trong index.js, gồm setup; không công bố tỷ lệ phần trăm khi chưa có mẫu số được định nghĩa. Thời gian chiến dịch khoảng 6.32 giây, tương đương 1.42 input/giây tính cả replay. Phép đo có setup token, Node startup, chuyển đổi coverage, solver và ghi evidence, không phải tốc độ thuần của target hoặc phép so sánh hiệu năng giữa các phương pháp.

4.4. Phân tích phản ví dụ theo luồng nghiệp vụ

Lộ tài khoản — TC-AUTH-3: request với email không tồn tại và request sai mật khẩu vào hai nhánh phản hồi khác nhau. Oracle so response công khai; kết quả FAIL cho thấy không đáp ứng yêu cầu chống enumeration trên mẫu. Timing là kênh riêng, cần phép đo kiểm soát nhiễu trước khi kết luận.

Sai kiểu/thiếu tiền — TC-ORD-3/4: giữ token, giỏ và shipping hợp lệ để tới OrderModel.save. Required hoặc ép kiểu làm phát sinh ngoại lệ, route bắt và trả HTTP 500. Concolic cung cấp input và trace nhánh tương ứng. Hướng sửa là kiểm tra kiểu/bắt buộc ở biên API, trả 4xx có kiểm soát; HTTP 500 chưa là lỗi bộ nhớ hoặc crash tiến trình.

Giá giả tự khớp — TC-ORD-18: catalog productId=1 có giá 200000, quantity=2; client gửi price=1 và totalAmount=2, không có phí/giảm giá. Nếu oracle chỉ tính từ body sẽ thấy 1×2=2 và bỏ sót. Đối chiếu catalog cho tổng mong đợi 400000; ứng dụng lưu tổng giả là vi phạm toàn vẹn. Giá âm và giá giả không tự đồng nghĩa integer overflow.

Trạng thái kết thúc — TC-OS-21: tạo đơn, hủy thành công rồi yêu cầu đang giao. Oracle xác nhận các bước chuẩn bị thành công, xem cả status HTTP và final_entity. Request cuối được chấp nhận cho thấy thiếu cưỡng chế cạnh chuyển trạng thái. Sử dụng version hợp lệ không thay thế kiểm tra vòng đời; đây không phải bằng chứng race condition.

Nâng quyền và mật khẩu — TC-USR-2/4: role=admin do client gửi được đánh giá bằng role, còn password_audit kiểm tra dữ liệu lưu, bcrypt và khả năng xác minh. Đăng nhập thành công không tự chứng minh password đã hash. Thiếu auth, IDOR và mass assignment là những kiểm soát khác nhau; hướng sửa là xác thực, phân quyền đối tượng, whitelist và hash trước lưu.

4.5. Kiểm tra độ đúng của bộ công cụ

Bộ hồi quy hiện có 28 bài đạt. Năm bài concolic mới kiểm tra: required=false cùng cast.empty=true là UNSAT; phủ định nhánh bỏ đúng hậu tố; missing khác null; mô hình sai/trace rỗng phải bị từ chối; cả tám đại diện được so với nhánh thực của API. Các bài còn lại kiểm tra oracle, truy vết ID, ngân sách, giao thức và điều phối. Kết quả này kiểm tra công cụ, không chứng minh toàn bộ ứng dụng an toàn.

controlled_comparison.py vẫn cho phép so mutation baseline và coverage feedback trên cùng target, seed, budget và môi trường. Phiên bản hiện tại chưa có bộ bằng chứng so sánh mới được đưa vào báo cáo này; không dùng số finding ở hai chiến dịch khác ngân sách/trạng thái để tuyên bố phương pháp nào tốt hơn. Cần chạy nhiều seed và quản lý nhiễu nếu đánh giá hiệu quả thống kê.

5. Thảo luận, hạn chế và hướng phát triển

5.1. Mức đáp ứng theo nhóm lỗi

Bảng 11. Thiết kế và mức thực nghiệm hiện tại

Nhóm lỗi	Mức hiện có	Chưa được khẳng định
Đầu vào / xử lý lỗi	Mutation, status oracle, trace/concolic totalAmount	Mọi injection, path traversal hoặc mọi đường lỗi
Quyền / toàn vẹn / logic	Có phản ví dụ quyền, giá và vòng đời	Lỗi gốc độc lập hoặc khả năng khai thác production
An toàn bộ nhớ	Thiết kế mẫu C và oracle sanitizer trong Excel	Buffer overflow, UAF, double-free hoặc leak runtime đã xác nhận
Số học	Có ràng buộc tiền và mô hình bổ sung	Integer overflow runtime từ mô hình int32
Race / timing	Có testcase trong Excel	Đã chạy đủ kịch bản đồng thời/timing trong campaign này

Không cần biến mọi testcase thành lỗi bộ nhớ để phù hợp môn học. Tuy nhiên, sự hiện diện của payload dài, mẫu C hoặc công thức SMT trong tài liệu không tự là bằng chứng lỗi runtime. Muốn xác nhận lỗi native cần build/instrument, driver, input, stderr/stack, mã thoát và đối chứng an toàn; leak có thể tồn tại khi tiến trình vẫn kết thúc bình thường.

5.2. Đe dọa tính hợp lệ của kết luận

- Mô phỏng datastore có thể khác Mongoose/MongoDB thật về ép kiểu, ràng buộc, transaction và concurrency; phải xác minh lại trước khi kết luận triển khai.
- Blackbox giữ dữ liệu còn khảo sát/concolic dùng tiến trình mới; replay từ trạng thái sạch không khôi phục mọi lịch sử thao tác và token/ID ban đầu.
- Coverage chỉ là dòng index.js có cả setup, chưa là tín hiệu nhánh/edge hoặc dấu vết truy cập bộ nhớ; 100% coverage cũng không bảo đảm hết lỗi.
- Concolic dùng mô hình thủ công và tám đại diện kiểu; kiểm tra chéo giảm nguy cơ sai mô hình nhưng không mở rộng kết luận ra mọi giá trị JavaScript.
- Ngân sách demo nhỏ, chưa có đánh giá ưu thế thống kê; chưa minimization input, corpus distillation hoặc dedup crash bằng stack trace.
- Các chỉ tiêu confirmed_memory_errors và confirmed_process_crashes là null vì chưa đo; không được chuyển thành 0 để tuyên bố không có lỗi.

5.3. Hướng khắc phục và phát triển

Ưu tiên ứng dụng: bảo vệ endpoint, phân quyền đối tượng, whitelist trường; tính giá theo catalog; kiểm tra kiểu/miền trước datastore; cưỡng chế trạng thái kết thúc; hash password và không lộ nội dung lỗi. Đây là đề xuất khắc phục, chưa trình bày như các bản vá đã được triển khai. Sau sửa cần chạy lại đúng input vi phạm và đối chứng hợp lệ để tạo bằng chứng trước/sau.

Ưu tiên công cụ tiếp theo: mở rộng seed và mutation có kiểm soát, tách coverage setup, lưu/nghiệm thu trạng thái trước request, giảm input lỗi, thực nghiệm race có bằng chứng lịch gửi/nhận, và bổ sung một thành phần native độc lập có sanitizer nếu nghiên cứu bộ nhớ. Chỉ mở rộng concolic sau khi mô hình/trace của vùng mới được kiểm tra chéo; không thêm solver rời rạc rồi gọi là symbolic toàn hệ thống.

6. Kết luận

Đề tài đã xây dựng luồng đặc tả → testcase/seed → fuzzing → oracle → bằng chứng trên API bán hàng mô phỏng. Bộ tài liệu giữ 104 testcase; campaign hiện thực thi 29 testcase API và lưu rõ 75 testcase ngoài phạm vi chạy. Kết quả có 9 testcase PASS, 20 FAIL trên mẫu/oracle, không ERROR trong 59 kịch bản.

Đóng góp của phiên bản hiện tại là phân biệt blackbox, khảo sát coverage và whitebox concolic có chọn lọc; liên kết trace nhánh totalAmount đang được API sử dụng với Z3 và lần chạy xác nhận. Bằng chứng mới cho thấy mô hình có thể sinh input đạt tiền tố khác và nhận biết tiền tố mâu thuẫn trong miền đại diện. Không tuyên bố đã kiểm chứng đủ 104 testcase, symbolic toàn bộ JavaScript hoặc xác nhận lỗi bộ nhớ production. Hướng tiếp theo là xác minh gần môi trường thật và đánh giá sau bản vá.

Tài liệu tham khảo

[1] Vũ Quang Dũng. Chương 1 (Phần 1): Giới thiệu về an toàn phần mềm. CSE703093, Trường Đại học Phenikaa. File Chuong1_Phan1_ATPM.pdf do người học cung cấp, 100 trang; không suy đoán năm xuất bản.

[2] Vũ Quang Dũng. Chương 4 (Phần 1): Phát hiện lỗ hổng phần mềm – Phân tích động. File Chuong4_Phan1_ATPM.pdf do người học cung cấp, 100 trang.

[3] Vũ Quang Dũng. Chương 4 (Phần 2): Sinh đường thực thi, blackbox và whitebox fuzzing. File Chuong4_Phan2_ATPM.pdf do người học cung cấp, 100 trang.

[4] Hướng dẫn Báo cáo Bài tập lớn: cấu trúc, định dạng và tiêu chí đánh giá. CSE703093. BTL_02_HuongDan_BaoCao (2).pdf do người học cung cấp; không có năm xuất bản được xác nhận.

[5] Hướng dẫn tạo Spec file dạng Excel theo template đặc tả. Hướng dẫn tạo Spec file dạng excel (2).pdf do người học cung cấp. Giữ sheet không đồng nghĩa đã thực thi toàn bộ pipeline trong hướng dẫn.

[6] Mã nguồn, package/requirements và 5 workbook nộp kèm; phiên bản thực nghiệm theo source_sha256 và spec_snapshot.json trong các thư mục kết quả được dẫn ở Phụ lục A.

[7] results/spec_excel_aligned/report.json và results/demo_chuong4_final/: dữ liệu thực nghiệm của báo cáo. Công thức, model, trace và replay của concolic nằm trong report/evidence tương ứng.

Phụ lục A. Cấu hình, lệnh chạy và bằng chứng

A.1. Chuẩn bị và chạy lại

Các lệnh sau chạy từ thư mục gốc dự án. Cần Python, Node.js/npm và dependency trong requirements/package-lock. Không cần mở server production hoặc MongoDB. Nên chọn thư mục output mới để không ghi đè bằng chứng cũ.

```
python -m pip install -r fuzzing-security-testing/requirements.txt
npm ci --prefix fuzzing-security-testing/node_harness
python -m unittest discover -s fuzzing-security-testing/tests -p "test_*.py"
```

```
python fuzzing-security-testing/fuzzers/run_ch4.py --seed 7 --fuzz-cases 8 --output
fuzzing-security-testing/results/spec_replay_new
```

```
python fuzzing-security-testing/fuzzers/run_fuzzing.py --iterations 42 --iterations-per-
case 8 --max-cases 2 --seed 7 --concolic
```

```
python fuzzing-security-testing/fuzzers/whitebox_fuzzer.py --concolic --iterations-per-
target 12
```

Có thể chạy node fuzzing-security-testing/node_harness/http_server.js ở terminal riêng rồi truyền --base-url http://127.0.0.1:5050 cho blackbox hoặc run_fuzzing. URL đó chỉ áp dụng bước blackbox; các bước khảo sát/concolic vẫn dùng Node harness cô lập. Không truyền URL thì công cụ tự mở/đóng HTTP harness.

Bảng 12. Bản đồ bằng chứng đang tồn tại

Nội dung	Đường dẫn từ fuzzing-security-testing/
Đặc tả và testcase	BaoCaoATPM/Module_*_spec.xlsx
Campaign + snapshot + excluded	results/spec_excel_aligned/
Demo tổng hợp	results/demo_chuong4_final/report.json
Input và response blackbox	results/demo_chuong4_final/blackbox/
Replay/lân cận/coverage	results/demo_chuong4_final/whitebox/
Concolic, model và trace	results/demo_chuong4_final/concolic/report.json; evidence/
Mã concolic và tracing	fuzzers/concolic.py; node_harness/branch_trace.js
Sổ tay trình bày	TRINH_BAY_TIEN_DO.md; SO_TAY_TESTCASE_VA_NGHIEP_VU.md

A.2. Chú ý khi tái hiện và sử dụng công cụ

Công cụ replay_failures.py kiểm tra hash code/workbook của campaign trước khi chạy; khi nguồn thay đổi phải tạo campaign mới. Kết quả lịch sử không được sửa hash để giả khớp phiên bản. Cần giữ helper server/concurrency.js theo đúng cấu trúc dự án dù campaign hiện tại không thực nghiệm đầy đủ concurrency.

Các công cụ/cấu hình CTL/SMT cũ đã được đưa khỏi bộ chính, nên không có lệnh verify_spec_workbooks.py trong phụ lục hiện tại. Concolic dùng z3-solver trực tiếp từ requirements.txt; đây là pipeline riêng, không phải bộ kiểm chứng toàn bộ sheet CTL/số học hoặc core.spec_pipeline của giảng viên.

Công cụ AI được sử dụng hỗ trợ rà soát, tổ chức code/tài liệu và biên soạn. Nhóm cần tự kiểm tra code, hiểu phương pháp, đối chiếu artifact và chịu trách nhiệm về nội dung nộp. Không dùng văn bản do công cụ sinh thay cho bằng chứng thực thi hoặc tự khai đã kiểm chứng phần chưa chạy.

Phụ lục B. Truy vết 29 testcase API đã chạy

Bảng dưới liên kết đúng ID trong Excel với kết quả campaign. Cột kịch bản là số execution thuộc ID, không là số lỗ hổng. Chi tiết từng input và oracle ở report.json; sổ tay chứa giải thích toàn bộ 104 testcase.

Bảng 13. Danh sách testcase API và kết quả theo ID

ID	Mục tiêu ngắn	Kịch bản	Kết quả
TC-AUTH-1	Xử lý an toàn trường xác thực bị thiếu hoặc rỗng	3	PASS
TC-AUTH-2	Ngoại lệ khi email/password sai kiểu dữ liệu	5	PASS
TC-AUTH-3	Rò rỉ sự tồn tại tài khoản qua phản hồi đăng nhập	1	FAIL
TC-AUTH-5	Chèn role vào input xác thực để nâng quyền	1	PASS
TC-AUTH-7	Khả năng xử lý email dài bất thường	1	PASS
TC-ORD-1	Sai kiểu và cấu trúc giỏ hàng đi vào tầng xử lý đơn	4	FAIL
TC-ORD-2	Thiếu cấu trúc lồng shippingInfo gây ngoại lệ	4	PASS
TC-ORD-3	Xử lý an toàn totalAmount thiếu/null và xác nhận nhánh required	1	FAIL
TC-ORD-4	Sai kiểu totalAmount: mutation và concolic xác nhận đường ép kiểu	12	FAIL
TC-ORD-5	Tin cậy giá từ client làm sai toàn vẹn đơn hàng	1	FAIL
TC-ORD-6	Giá trị âm vượt qua kiểm tra miền số tiền	1	FAIL
TC-ORD-18	Giả mạo đồng thời đơn giá và tổng tiền để vượt oracle	1	FAIL
TC-ORD-19	Đối chứng seed hợp lệ trước khi đột biến đơn hàng	1	PASS
TC-OS-1	Vượt quyền cập nhật tài nguyên đơn hàng	2	FAIL
TC-OS-2	Giá trị enum bất thường đi vào xử lý trạng thái	2	FAIL
TC-OS-3	Phá vỡ bất biến sau trạng thái đã giao	1	FAIL

ID	Mục tiêu ngắn	Kịch bản	Kết quả
TC-OS-4	ID hoặc status sai kiểu gây lỗi tầng xử lý	2	FAIL
TC-OS-21	Phá vỡ trạng thái kết thúc sau khi hủy đơn	1	FAIL
TC-OS-5	Đối chứng luồng và tác dụng phụ thông báo	1	PASS
TC-PRD-1	Vượt quyền tạo hoặc sửa dữ liệu sản phẩm	4	FAIL
TC-PRD-2	Giá âm vượt qua ràng buộc miền dữ liệu	1	FAIL
TC-PRD-3	Giá sản phẩm sai kiểu gây ngoại lệ xử lý	1	FAIL
TC-PRD-5	ID trùng gây ghi đè hoặc mơ hồ định danh sản phẩm	1	FAIL
TC-PRD-4	Đối chứng lỗi truy cập sản phẩm không tồn tại	1	PASS
TC-USR-1	Truy cập trái phép hồ sơ tài khoản khác	2	FAIL
TC-USR-2	Mass assignment làm thay đổi quyền tài khoản	1	FAIL
TC-USR-4	Dữ liệu mật khẩu nhạy cảm bị lưu dạng rõ	1	FAIL
TC-USR-3	ID sai định dạng gây ngoại lệ truy vấn hồ sơ	1	FAIL
TC-USR-5	Vượt cơ chế bảo vệ khởi tạo tài khoản admin	1	PASS